

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.manifold import TSNE
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, Flatten, Reshape
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.datasets import mnist

# Load MNIST dataset
(X_train_full, y_train_full), (X_test, y_test) = mnist.load_data()

# Normalize data to [0, 1]
X_train_full = X_train_full / 255.0
X_test = X_test / 255.0

# Flatten images (28x28 -> 784)
X_train_full = X_train_full.reshape(-1, 28 * 28)
X_test = X_test.reshape(-1, 28 * 28)

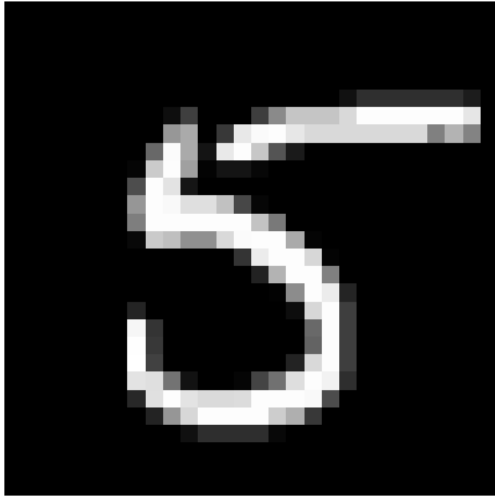
# Split training data into training and validation sets
X_train, X_val, y_train, y_val = train_test_split(X_train_full, y_train_full, test_size=0.2, random_state=42)

# Print starting samples
print("Starting Samples:")
for i in range(5):
    plt.imshow(X_train[i].reshape(28, 28), cmap='gray')
    plt.title(f"Label: {y_train[i]}")
    plt.axis('off')
    plt.show()
```

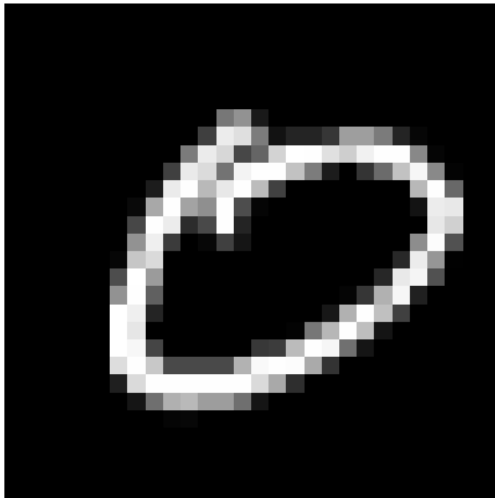


11490434/11490434 0s 0us/step
Starting Samples:

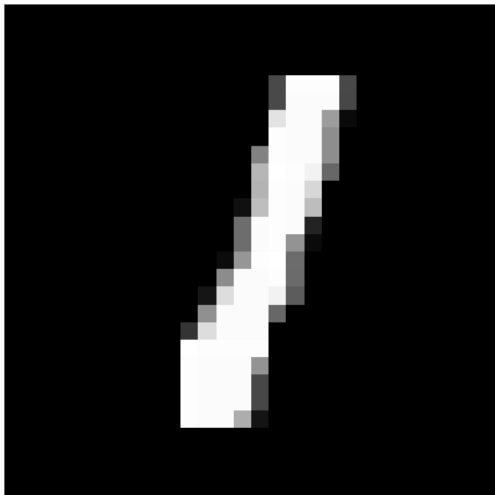
Label: 5



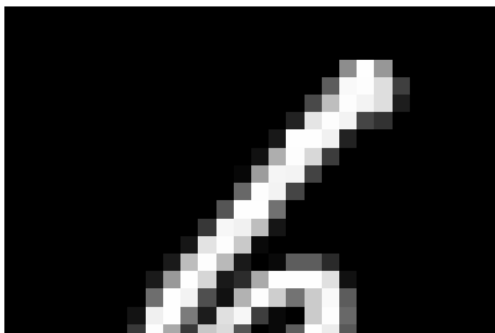
Label: 0



Label: 1

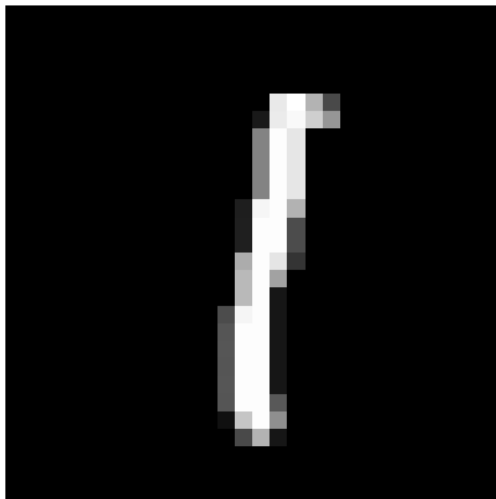


Label: 6





Label: 1



```
# Input layer
input_dim = X_train.shape[1] # 784
input_layer = Input(shape=(input_dim,))

# Encoder
encoded = Dense(128, activation='relu')(input_layer)
encoded = Dense(64, activation='relu')(encoded)
encoded = Dense(32, activation='relu')(encoded) # Latent space

# Decoder
decoded = Dense(64, activation='relu')(encoded)
decoded = Dense(128, activation='relu')(decoded)
decoded = Dense(input_dim, activation='sigmoid')(decoded)

# Autoencoder model
autoencoder = Model(input_layer, decoded)

# Encoder model (to extract latent space)
encoder = Model(input_layer, encoded)

# Compile the model
autoencoder.compile(optimizer=Adam(), loss='mse')

# Print model summary
autoencoder.summary()
```

Model: "functional_2"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 784)	0
dense_5 (Dense)	(None, 128)	100,480
dense_6 (Dense)	(None, 64)	8,256
dense_7 (Dense)	(None, 32)	2,080
dense_8 (Dense)	(None, 64)	2,112
dense_9 (Dense)	(None, 128)	8,320
dense_10 (Dense)	(None, 784)	101,136

Total params: 222,384 (868.69 KB)

```
# Train the autoencoder
history = autoencoder.fit(
    X_train, X_train,
    epochs=20,
    batch_size=256,
    validation_data=(X_val, X_val),
```

```

    verbose=2
)

Epoch 1/20
188/188 - 6s - 34ms/step - loss: 0.0687 - val_loss: 0.0417
Epoch 2/20
188/188 - 1s - 4ms/step - loss: 0.0342 - val_loss: 0.0287
Epoch 3/20
188/188 - 1s - 4ms/step - loss: 0.0261 - val_loss: 0.0241
Epoch 4/20
188/188 - 1s - 4ms/step - loss: 0.0229 - val_loss: 0.0218
Epoch 5/20
188/188 - 1s - 6ms/step - loss: 0.0206 - val_loss: 0.0197
Epoch 6/20
188/188 - 1s - 3ms/step - loss: 0.0189 - val_loss: 0.0185
Epoch 7/20
188/188 - 1s - 3ms/step - loss: 0.0178 - val_loss: 0.0173
Epoch 8/20
188/188 - 1s - 3ms/step - loss: 0.0168 - val_loss: 0.0166
Epoch 9/20
188/188 - 1s - 4ms/step - loss: 0.0160 - val_loss: 0.0160
Epoch 10/20
188/188 - 1s - 3ms/step - loss: 0.0154 - val_loss: 0.0152
Epoch 11/20
188/188 - 1s - 3ms/step - loss: 0.0148 - val_loss: 0.0146
Epoch 12/20
188/188 - 1s - 3ms/step - loss: 0.0142 - val_loss: 0.0142
Epoch 13/20
188/188 - 1s - 3ms/step - loss: 0.0138 - val_loss: 0.0137
Epoch 14/20
188/188 - 1s - 3ms/step - loss: 0.0134 - val_loss: 0.0134
Epoch 15/20
188/188 - 1s - 3ms/step - loss: 0.0130 - val_loss: 0.0129
Epoch 16/20
188/188 - 1s - 4ms/step - loss: 0.0127 - val_loss: 0.0127
Epoch 17/20
188/188 - 1s - 3ms/step - loss: 0.0123 - val_loss: 0.0124
Epoch 18/20
188/188 - 1s - 3ms/step - loss: 0.0121 - val_loss: 0.0122
Epoch 19/20
188/188 - 1s - 3ms/step - loss: 0.0119 - val_loss: 0.0120
Epoch 20/20
188/188 - 1s - 3ms/step - loss: 0.0117 - val_loss: 0.0119

```

```

# Evaluate on test set
test_loss = autoencoder.evaluate(X_test, X_test, verbose=0)
print(f"Test Loss (MSE): {test_loss:.4f}")

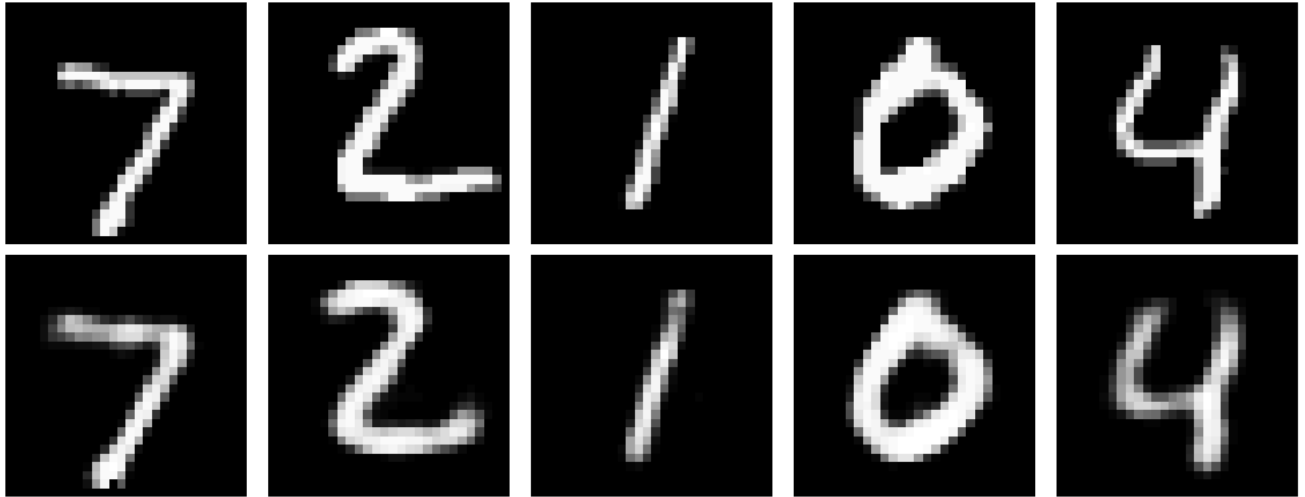
# Visualize original vs reconstructed images
reconstructed = autoencoder.predict(X_test)

print("Original and Reconstructed Samples:")
num_images = 5
plt.figure(figsize=(10, 4))
for i in range(num_images):
    # Original
    plt.subplot(2, num_images, i + 1)
    plt.imshow(X_test[i].reshape(28, 28), cmap='gray')
    plt.axis('off')
    if i == 0:
        plt.ylabel("Original")

    # Reconstructed
    plt.subplot(2, num_images, i + 1 + num_images)
    plt.imshow(reconstructed[i].reshape(28, 28), cmap='gray')
    plt.axis('off')
    if i == 0:
        plt.ylabel("Reconstructed")
plt.tight_layout()
plt.show()

```

Test Loss (MSE): 0.0115
 WARNING:tensorflow:5 out of the last 15 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distri
 313/313 ————— 1s 2ms/step
 Original and Reconstructed Samples:

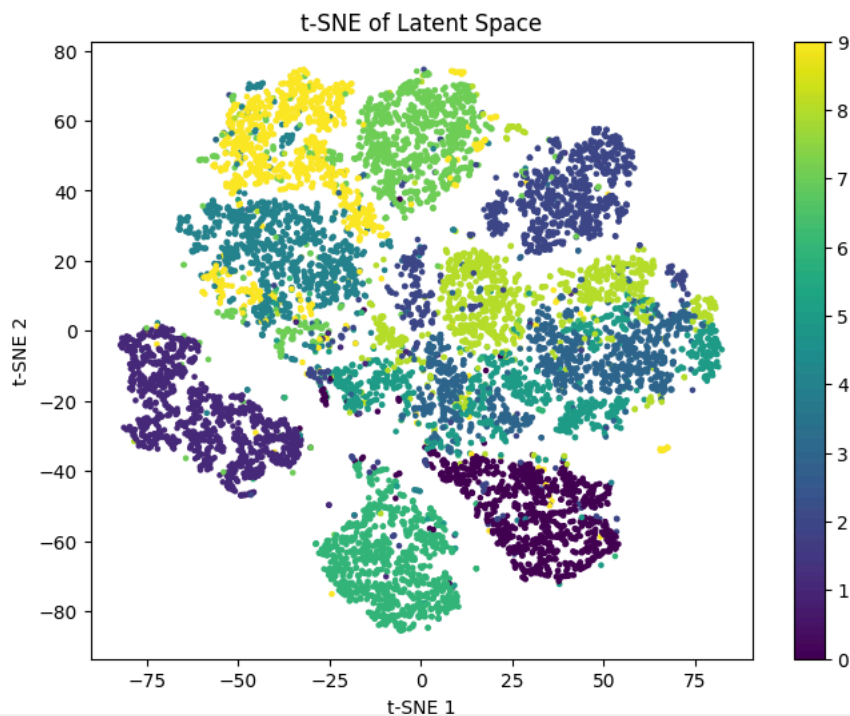


```
# Extract latent representations
latent_representations = encoder.predict(X_test)

# Visualize latent space using t-SNE
tsne = TSNE(n_components=2, random_state=42)
latent_2d = tsne.fit_transform(latent_representations)

plt.figure(figsize=(8, 6))
plt.scatter(latent_2d[:, 0], latent_2d[:, 1], c=y_test, cmap='viridis', s=5)
plt.colorbar()
plt.title("t-SNE of Latent Space")
plt.xlabel("t-SNE 1")
plt.ylabel("t-SNE 2")
plt.show()
```

313/313 ————— 1s 1ms/step



Start coding or [generate](#) with AI.

```
!pip install tensorflow-datasets matplotlib scikit-learn
```

```

from sklearn.datasets import fetch_lfw_people
import matplotlib.pyplot as plt

# Load the LFW dataset
lfw_people = fetch_lfw_people(min_faces_per_person=70, resize=0.4)

# Get the images and labels
X = lfw_people.images
y = lfw_people.target

# Plot some example images
num_images = 16
plt.figure(figsize=(10, 10))
for i in range(num_images):
    plt.subplot(4, 4, i + 1)
    plt.imshow(X[i], cmap='gray')
    plt.title(f"Label: {lfw_people.target_names[y[i]]}")
    plt.axis('off')
plt.suptitle("Sample Images from LFW Dataset")
plt.show()

```



Sample Images from LFW Dataset

Label: Hugo Chavez



Label: Tony Blair



Label: George W Bush



Label: Colin Powell



Label: Ariel Sharon



Label: Colin Powell



Label: George W Bush Label: Gerhard Schroeder



Label: George W Bush



Label: Ariel Sharon



Label: George W Bush



Label: Donald Rumsfeld



Label: George W Bush



Label: Colin Powell



Label: Gerhard Schroeder Label: George W Bush



```

from sklearn.datasets import fetch_lfw_people
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
from sklearn.manifold import TSNE

```

```

# Load the LFW dataset
lfw_people = fetch_lfw_people(min_faces_per_person=70, resize=0.4)
X = lfw_people.images

```

```

y = lfw_people.target
X_flat = X.reshape(X.shape[0], -1) # Flatten the images for the neural network

# Train-test split
X_train, X_test = train_test_split(X_flat, test_size=0.2, random_state=42)

# Build the autoencoder model
input_layer = layers.Input(shape=(X_train.shape[1],))
encoder = layers.Dense(128, activation='relu')(input_layer)
encoder = layers.Dense(64, activation='relu')(encoder)
decoder = layers.Dense(128, activation='relu')(encoder)
output_layer = layers.Dense(X_train.shape[1], activation='sigmoid')(decoder)

autoencoder = models.Model(input_layer, output_layer)

# Compile the model
autoencoder.compile(optimizer='adam', loss='mse')

# Train the autoencoder
history = autoencoder.fit(X_train, X_train, epochs=50, batch_size=256, validation_data=(X_test, X_test))

# Evaluate the model on the testing set
test_loss = autoencoder.evaluate(X_test, X_test)
print(f"Test loss (Reconstruction error): {test_loss}")

```

```

Epoch 23/50
5/5 ————— 0s 10ms/step - loss: 0.0156 - val_loss: 0.0155
Epoch 24/50
5/5 ————— 0s 8ms/step - loss: 0.0156 - val_loss: 0.0155
Epoch 25/50
5/5 ————— 0s 8ms/step - loss: 0.0155 - val_loss: 0.0153
Epoch 26/50
5/5 ————— 0s 8ms/step - loss: 0.0151 - val_loss: 0.0151
Epoch 27/50
5/5 ————— 0s 8ms/step - loss: 0.0150 - val_loss: 0.0152
Epoch 28/50
5/5 ————— 0s 8ms/step - loss: 0.0151 - val_loss: 0.0150
Epoch 29/50
5/5 ————— 0s 8ms/step - loss: 0.0148 - val_loss: 0.0147
Epoch 30/50
5/5 ————— 0s 9ms/step - loss: 0.0146 - val_loss: 0.0148
Epoch 31/50
5/5 ————— 0s 8ms/step - loss: 0.0147 - val_loss: 0.0145
Epoch 32/50
5/5 ————— 0s 8ms/step - loss: 0.0145 - val_loss: 0.0147
Epoch 33/50
5/5 ————— 0s 8ms/step - loss: 0.0147 - val_loss: 0.0155
Epoch 34/50
5/5 ————— 0s 8ms/step - loss: 0.0150 - val_loss: 0.0148
Epoch 35/50
5/5 ————— 0s 8ms/step - loss: 0.0144 - val_loss: 0.0142
Epoch 36/50
5/5 ————— 0s 9ms/step - loss: 0.0142 - val_loss: 0.0141
Epoch 37/50
5/5 ————— 0s 13ms/step - loss: 0.0139 - val_loss: 0.0139
Epoch 38/50
5/5 ————— 0s 9ms/step - loss: 0.0137 - val_loss: 0.0139
Epoch 39/50
5/5 ————— 0s 10ms/step - loss: 0.0138 - val_loss: 0.0149
Epoch 40/50
5/5 ————— 0s 8ms/step - loss: 0.0144 - val_loss: 0.0143
Epoch 41/50
5/5 ————— 0s 8ms/step - loss: 0.0139 - val_loss: 0.0146
Epoch 42/50
5/5 ————— 0s 8ms/step - loss: 0.0140 - val_loss: 0.0138
Epoch 43/50
5/5 ————— 0s 8ms/step - loss: 0.0137 - val_loss: 0.0139
Epoch 44/50
5/5 ————— 0s 8ms/step - loss: 0.0137 - val_loss: 0.0139
Epoch 45/50
5/5 ————— 0s 8ms/step - loss: 0.0135 - val_loss: 0.0135
Epoch 46/50
5/5 ————— 0s 8ms/step - loss: 0.0133 - val_loss: 0.0135
Epoch 47/50
5/5 ————— 0s 8ms/step - loss: 0.0133 - val_loss: 0.0141
Epoch 48/50
5/5 ————— 0s 8ms/step - loss: 0.0135 - val_loss: 0.0136
Epoch 49/50
5/5 ————— 0s 8ms/step - loss: 0.0133 - val_loss: 0.0140
Epoch 50/50
5/5 ————— 0s 8ms/step - loss: 0.0138 - val_loss: 0.0138
9/9 ————— 0s 2ms/step - loss: 0.0136
Test loss (Reconstruction error): 0.013772033154964447

```

```

# Visualize the reconstruction error
reconstructed_images = autoencoder.predict(X_test)

```

```

reconstruction_error = np.mean(np.square(X_test - reconstructed_images), axis=1)

plt.figure(figsize=(10, 5))
plt.hist(reconstruction_error, bins=50, color='blue', alpha=0.7)
plt.title('Reconstruction Error Distribution')
plt.xlabel('Reconstruction Error')
plt.ylabel('Frequency')
plt.show()

# Visualize the comparison between original and reconstructed data for the first image in the test set
plt.figure(figsize=(10, 5))

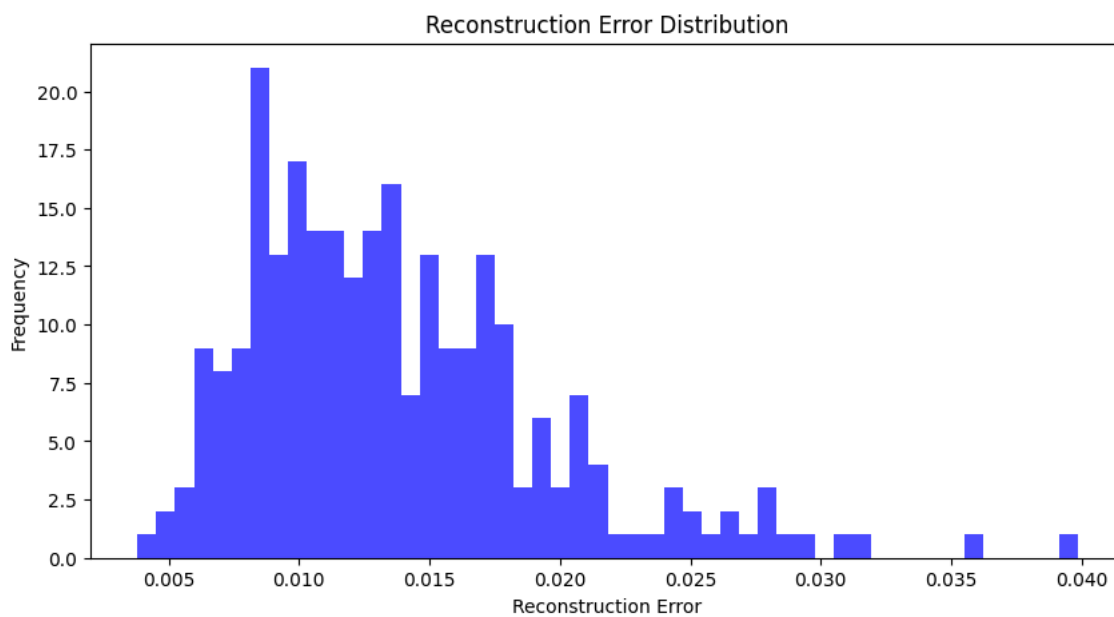
plt.subplot(1, 2, 1)
plt.imshow(X_test[0].reshape(50, 37), cmap='gray')
plt.title("Original Image")
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(reconstructed_images[0].reshape(50, 37), cmap='gray')
plt.title("Reconstructed Image")
plt.axis('off')

plt.show()

```

9/9 — 0s 12ms/step



Original Image



Reconstructed Image



```

# Extract the latent space (the output of the encoder)
encoder_model = models.Model(input_layer, encoder)
latent_space = encoder_model.predict(X_test)

```

```

# Visualize the latent space using t-SNE
tsne = TSNE(n_components=2, random_state=42)
latent_2d = tsne.fit_transform(latent_space)

```